

dd.txtのデータはシングルコーテーション'で囲まれた要素がコンマ,で区切られている。要素は文字列のダイアログとuint64_tのプロパティが必ず交互に並んでいる。ただし、uint64_tは0xfffffffffffffffff



データ追加・削除が発生せず、プロトタイプとしてシンプルな実装が適しているため、データコンテナAには `std::vector` と構造体（`std::pair` または独自構造体）の組み合わせが最適です。

要素数が固定であり一対一対応のペアであるため、

`struct Element { std::string text; uint64_t prop; };` を定義し、`std::vector<Element>` に格納します。16進数文字列から `uint64_t` への変換には `std::stoull(str, nullptr, 16)` を使用します。

C++



```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>

// データのペア構造体
struct Element {
    std::string text;
    uint64_t prop;
};

// プロトタイプのためグローバルに配置
std::vector<Element> containerA;
std::vector<Element> containerB;

int main() {
    std::ifstream file("dd.txt");
    if (!file.is_open()) {
        std::cerr << "エラー: dd.txt を開けませんでした。" << std::endl;
        return 1;
    }
}
```

```

std::string token;
bool is_text = true; // テキストとプロパティが交互に現れるフラグ
std::string current_text;

// クォート ' ' で区切って読み込み
while (std::getline(file, token, '\\')) {
    // 余白やカンマ等の不要な区切り文字をスキップ
    if (token.empty() || token == "," || token == " " || token == "\\n")
        continue;
    }

    if (is_text) {
        current_text = token;
        is_text = false;
    } else {
        // 16進数文字列 (例: "0xff8c") を uint64_t に変換
        uint64_t prop = std::stoull(token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}
file.close();

// 例: コンテナAを走査し、特定のビット条件 (例: 0x80 ビットが立っている) でコンテナI
uint64_t mask = 0x80;
for (const auto& item : containerA) {
    if ((item.prop & mask) != 0) {
        containerB.push_back(item);
    }
}

// 処理の最後にコンテナAの全要素を出力
std::cout << "--- Container A Elements ---" << std::endl;
for (size_t i = 0; i < containerA.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerA[i].text
        << ", Prop: 0x" << std::hex << containerA[i].prop << std::dec
        << "\n";
}

return 0;
}

```

上の処理をSDL_MAIN_USE_CALLBACKS 1の中に組み込んでほしい。

SDL_MAIN_USE_CALLBACKS 1 を使用する場合、従来の `int main()` は使用せず、SDL3が提供するコールバック関数（`SDL_AppInit` , `SDL_AppIterate` , `SDL_AppEvent` , `SDL_AppQuit` ）に処理を分散させます。

データ読み込みと抽出処理は初期化フェーズである `SDL_AppInit` 内で実行し、描画やメインループを行わない最小限のプロトタイプ構成として記述します。

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>

// データのペア構造体
struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;

// アプリケーション初期化（従来の main 関数の開始部分に相当）
SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    std::ifstream file("dd.txt");
    if (!file.is_open()) {
        std::cerr << "エラー: dd.txt を開けませんでした。" << std::endl;
        return SDL_APP_FAILURE;
    }

    std::string token;
    bool is_text = true;
    std::string current_text;
```

```

// クォート ' ' で区切って読み込み
while (std::getline(file, token, '\\')) {
    if (token.empty() || token == "," || token == " " || token == "\\n")
        continue;
    }

    if (is_text) {
        current_text = token;
        is_text = false;
    } else {
        uint64_t prop = std::stoull(token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}
file.close();

// ビットマスクによるコンテナBへの抽出 (例: 0x80 ビット判定)
uint64_t mask = 0x80;
for (const auto& item : containerA) {
    if ((item.prop & mask) != 0) {
        containerB.push_back(item);
    }
}

// コンテナAの要素を出力
std::cout << "--- Container A Elements ---" << std::endl;
for (size_t i = 0; i < containerA.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerA[i].text
        << ", Prop: 0x" << std::hex << containerA[i].prop << std::
}

// 処理完了後に即時終了する場合は SDL_APP_SUCCESS を返して SDL_AppQuit へ遷移
return SDL_APP_SUCCESS;
}

// イベント処理 (本プロトタイプでは未使用)
SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }
    return SDL_APP_CONTINUE;
}

// 毎フレームのループ処理 (ワンショット処理のため即終了)
SDL_AppResult SDL_AppIterate(void *appstate) {
    // コンソール出力処理が完了しているため、そのままアプリを正常終了
    return SDL_APP_SUCCESS;
}

```

```
// 終了処理
void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    // 解放処理等（必要に応じて記述）
}
```

main.cppと同じ階層にdd.txtがあるんだけど、なぜか開けないって。

SDL_MAIN_USE_CALLBACKS 1 (SDL3) を使用している環境で dd.txt が開けない場合、実行時の「カレントディレクトリ (作業ディレクトリ)」が main.cpp のある場所とズレていることが原因である可能性が極めて高いです。

ビルド環境やOSごとの発生原因と、具体的な対処法をまとめました。

よくある原因と対処法

1. **IDE (Visual Studio / VS Code 等) の作業ディレクトリ設定**
IDEから実行ボタンを押して起動した場合、デフォルトの作業ディレクトリが main.cpp のある場所ではなく、ビルド出力先 (x64/Debug/ や build/ など) に設定されていることがあります。

Visual Studio の場合:

プロジェクトのプロパティ > 配置 / デバッグ > 「作業ディレクトリ」を \$(ProjectDir) に設定します。

VS Code の場合:

.vscode/launch.json の "cwd" を "\${workspaceFolder}" に設定します。

2. **SDL3のパス取得関数 SDL_GetBasePath() を使う (一番確実に解決する方法)**
実行ファイルの置かれているディレクトリ (パス) をSDLのAPIで取得し、絶対パスを作成してファイルを開くことで、カレントディレクトリに依存しない実装にできます。

修正コード例（SDL_GetBasePath() を使用）

SDL_AppInit 内のファイルオープン処理を以下のように変更してください。

C++



```
SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    // 実行ファイルが存在するディレクトリのパスを取得（例: "C:/path/to/exe/"）
    const char *base_path = SDL_GetBasePath();
    std::string file_path;

    if (base_path) {
        file_path = std::string(base_path) + "dd.txt";
    } else {
        // パスが取得できない場合のフォールバック
        file_path = "dd.txt";
    }

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::endl;
        std::cerr << "現在の検索パスを確認してください。" << std::endl;
        return SDL_APP_FAILURE;
    }

    // --- 以降の読み込み処理は同じ ---
}
```

※ビルド後の実行ファイル（.exe など）が main.cpp と同じ場所に生成されない設定になっている場合は、dd.txt をビルド出力先ディレクトリ（.exe が配置される場所）にも配置するか、file_path に相対パスを指定して調整してください。

どうしてこのような結果になると思う？ --- Container A Elements ---

[0] Text: konichiwa, Prop: 0x80

[1] Text: ohayo, Prop: 0x0



仰る通り、ロジック上は `ohayo` (`0x0`) ではなく `arigato` (`0x180`) がコンテナBに入るのが正しい結果です。

`0x0 & 0x80` は `0` になるため条件を満たさず、`0x180 & 0x80` は `0x80` (非ゼロ) になるため条件を満たすはずですが、

それにもかかわらず `ohayo` が入ってしまった原因は、`dd.txt` のパース処理でカンマや空白の扱いによりデータのインデックスがズレてしまい、`text` と `prop` のペアが崩れていた可能性が極めて高いです。

原因のメカニズム（なぜズレたのか）

元の読み込みロジックは、`'` (シングルクォート) を区切り文字として処理していました。

C++



```
while (std::getline(file, token, '\'')) { ... }
```

もし `dd.txt` が以下のようなフォーマットだった場合：

Plaintext



```
'konichiwa', '0x80', 'ohayo', '0x0', 'arigato', '0x180'
```

`getline` で `'` を区切りに分解すると、取得されるトークン列は次のようになります。

1. `""` (先頭のクォートの前)
2. `"konichiwa"` (テキスト)
3. `", "` (クォートとクォートの間のカンマ+空白) ★ここが原因
4. `"0x80"` (プロパティ)
5. `", "` (カンマ+空白) ★

6. "ohayo" (テキスト)

...

コード内のスキップ条件 `token == ", "` が、実際のファイルのスペースの数（例: カンマの後にスペースがない `", "` や、タブ文字、改行コードなど）と少しでもズレていると、`", "` という文字列自体が `text` として読み込まれ、フラグの交互切り替え（`is_text`）が反転してズレが生じます。

その結果、`stoull` がパースに失敗して `0` になったり、文字列と数値の組み合わせが1つずつズレて `ohayo` のプロパティが誤判定されたりします。

解決策：堅牢なパース処理への修正

区切り文字や余白に依存しないよう、「英数字や16進数のトークンだけを順番に抽出する」方式に変更すると、改行やスペースの入る形式がどうであれ確実に正しく読み込めます。

`SDL_AppInit` 内の読み込み部分を以下のように修正してください。

C++



```
std::string token;
bool is_text = true;
std::string current_text;

// シングルクォート（'）で区切ってトークンを取得
while (std::getline(file, token, '\\')) {
    // トークン前後の空白や改行をトリムする
    size_t first = token.find_first_not_of(" \t\r\n,");
    if (first == std::string::npos) {
        continue; // カンマや空白だけのトークンはスキップ
    }
    size_t last = token.find_last_not_of(" \t\r\n,");
    std::string clean_token = token.substr(first, (last - first + 1));

    if (is_text) {
        current_text = clean_token;
        is_text = false;
    } else {
        try {
            uint64_t prop = std::stoull(clean_token, nullptr, 16);
            containerA.push_back({current_text, prop});
        } catch (...) {
            std::cerr << "変換エラー: " << clean_token << std::endl;
        }
    }
}
```

```

    }
    is_text = true;
}
}

```

この修正を入れることでペアのズレがなくなり、意図通り `arigato` (`0x180`) がコンテナBに抽出されるようになります。

ちなみに元データです。'konichiwa', '0x80', 'ohayo', '0x0', 'arigato', '0x180'

元データがその通りの配置だと、やはり `std::getline(file, token, '\\')` で分解した際のトークン分解挙動が直接の原因です。

実際に元のコードで `,` を区切り文字として読み込むと、以下のような順序でトークンが処理されていました。

Plaintext



元データ: 'konichiwa', '0x80', 'ohayo', '0x0', 'arigato', '0x180'

1. トークン 0: "" (先頭の `,` より前の空文字列) → スキップ
2. トークン 1: "konichiwa" → `current_text` に保存 (`is_text = false`)
3. トークン 2: ", " → スキップ処理にかかりスキップ (※ここまでは正常)
4. トークン 3: "0x80" → `stoull` で変換され、`konichiwa`, `0x80` がコンテナ Aに入る (`is_text = true`)
5. トークン 4: ", " → スキップ処理
6. トークン 5: "ohayo" → `current_text` に保存 (`is_text = false`)
7. トークン 6: ", " → 【ここで問題発生】
もしファイルのカンマの後にスペースが2つ入っていたり、タブや改行が

入っていたりすると `token == ", "` の一致に失敗します。
すると `", "` がテキストとして処理され、以降の `is_text` フラグ（偶数・奇数の交互判定）が完全に反転してしまいます。

フラグが反転すると、本来「数値」であるはずの `"0x0"` や `"0x180"` をテキストとして扱い、逆に「テキスト」を数値として変換しようとしてパースが崩壊します。

原因を根本解決するシンプルな書き方

`getline` でクォート区切りに頼るよりも、文字列からクォート `'` とカンマ `,` を取り除きながらデータ（`std::string`）を1つずつ順番に流し込む方式が一番安全で確実です。

C++ の `std::stringstream` や `>>` 演算子を使えば、余計な空白・改行・カンマを自動で飛び越えてくれます。

確実に動作する読み込み処理

C++



```
#include <sstream>
```

```
// ...（前略）...
```

```
SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {  
    std::ifstream file("dd.txt");  
    if (!file.is_open()) {  
        return SDL_APP_FAILURE;  
    }  
  
    std::string token;  
    std::string current_text;  
    bool is_text = true;  
  
    // ファイルから空白区切りで単語（トークン）を1つずつ読み込む  
    while (file >> token) {  
        // トークンに含まれる ' や , や 空白 をすべて削除する  
        std::string clean_token = "";  
        for (char c : token) {  
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')  
                clean_token += c;  
        }  
    }  
}
```

```

// 中身が空になったトークンは無視
if (clean_token.empty()) continue;

if (is_text) {
    current_text = clean_token;
    is_text = false; // 次はプロパティ (数値)
} else {
    uint64_t prop = std::stoull(clean_token, nullptr, 16);
    containerA.push_back({current_text, prop});
    is_text = true; // 次はテキスト
}
}
file.close();

// 抽出処理 (0x80のビット判定)
uint64_t mask = 0x80;
for (const auto& item : containerA) {
    if ((item.prop & mask) != 0) {
        containerB.push_back(item);
    }
}

// 出力確認
std::cout << "--- Container A Elements ---" << std::endl;
for (size_t i = 0; i < containerA.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerA[i].text
                << ", Prop: 0x" << std::hex << containerA[i].prop << std::dec
    << std::endl;
}

std::cout << "--- Container B Elements ---" << std::endl;
for (size_t i = 0; i < containerB.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerB[i].text
                << ", Prop: 0x" << std::hex << containerB[i].prop << std::dec
    << std::endl;
}

return SDL_APP_SUCCESS;
}

```

この方法であれば、スペースの数や改行の有無に左右されず、順番通りに
[konichiwa: 0x80], [ohayo: 0x0], [arigato: 0x180] とペアが作成され、
コンテナBには想定通り konichiwa と arigato のみが抽出されます。

excludes 56-63
 requires 48-55
 unlocks 40-47
 scene 36-39

▼

各プロパティ項目が64ビットのどの範囲に配置されているかを踏まえ、ビットシフト（>>）とマスク演算（&）を行うことで条件判定を実装します。

ビット定義とマスク条件の算出

scene [36-39]: 4ビット幅（0xF）→ $(prop \gg 36) \& 0xF$

newRoot [16-23]: 8ビット幅（0xFF）→ $(prop \gg 16) \& 0xFF$

requires [48-55]: 8ビット幅（0xFF）→ $(prop \gg 48) \& 0xFF$

excludes [56-63]: 8ビット幅（0xFF）→ $(prop \gg 56) \& 0xFF$

hasRead [34]: 1ビット（1）→ $(prop \gg 34) \& 1$

2の条件（gUnlocks と requires をマスクした結果が requires と一致する）は、requires に要求される全ビットが gUnlocks 内で立っているかを判定することを意味するため、 $(gUnlocks \& req) == req$ と表現できます。

完全なプログラムコード

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>

struct Element {
    std::string text;
    uint64_t prop;
};
```

```

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;

// グローバルな条件判定用変数
unsigned gScene = 0;           // 0で初期化
unsigned gRoot = 0;           // 比較用のRoot値（必要に応じて変更）
unsigned gUnlocks = 0;        // 解放済みフラグ用ビットマスク（必要に応じて変更）

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    // -----
    // 1. dd.txt から containerA へのファイル読み込み
    // -----
    const char *base_path = SDL_GetBasePath();
    std::string file_path = base_path ? std::string(base_path) + "dd.txt" :

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;
    bool is_text = true;

    while (file >> token) {
        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }

        if (clean_token.empty()) continue;

        if (is_text) {
            current_text = clean_token;
            is_text = false;
        } else {
            uint64_t prop = std::stoull(clean_token, nullptr, 16);
            containerA.push_back({current_text, prop});
            is_text = true;
        }
    }
    file.close();

    // -----
    // 2. containerA から containerB への抽出

```

```

// 条件: scene [36-39] が gScene と一致すること
// -----
for (const auto& item : containerA) {
    unsigned scene = (item.prop >> 36) & 0xF; // 4ビット幅
    if (scene == gScene) {
        containerB.push_back(item);
    }
}

// -----
// 3. containerB から containerC への抽出
// 条件1: root [16-23] == gRoot
// 条件2: (gUnlocks & requires) == requires
// 条件3: hasRead [34] == 0
// 条件4: (gUnlocks & excludes) == 0 (false)
// -----
for (const auto& item : containerB) {
    uint64_t prop = item.prop;

    unsigned root      = (prop >> 16) & 0xFF; // 8ビット幅 (newRoot)
    unsigned hasRead   = (prop >> 34) & 0x1;  // 1ビット
    unsigned requires  = (prop >> 48) & 0xFF; // 8ビット幅
    unsigned excludes  = (prop >> 56) & 0xFF; // 8ビット幅

    // 条件判定
    bool cond1 = (root == gRoot);
    bool cond2 = ((gUnlocks & requires) == requires);
    bool cond3 = (hasRead == 0);
    bool cond4 = ((gUnlocks & excludes) == 0);

    if (cond1 && cond2 && cond3 && cond4) {
        containerC.push_back(item);
    }
}

// -----
// 4. 結果出力 (確認用)
// -----
std::cout << "--- Container A (" << containerA.size() << " items) ---" <
for (size_t i = 0; i < containerA.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerA[i].text
                << ", Prop: 0x" << std::hex << containerA[i].prop << std::
}

std::cout << "\n--- Container B (" << containerB.size() << " items) ---"
for (size_t i = 0; i < containerB.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerB[i].text
                << ", Prop: 0x" << std::hex << containerB[i].prop << std::
}

```

```

std::cout << "\n--- Container C (" << containerC.size() << " items) ---"
for (size_t i = 0; i < containerC.size(); ++i) {
    std::cout << "[" << i << "] Text: " << containerC[i].text
                << ", Prop: 0x" << std::hex << containerC[i].prop << std::
}

return SDL_APP_SUCCESS;
}

SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }
    return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppIterate(void *appstate) {
    return SDL_APP_SUCCESS;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
}

```

ビットマスク定義を標準化するためのヘッダーやマクロ関数を作成しますか？

↪ はい

なにが
 だめなのかさっぱり
 だ。/home/genios/Projects/NEELA/DDMEditor/main.cpp:85:1
 8: error: expected unqualified-id before 'requires'

原因は `requires` が **C++20** から導入された予約語（キーワード）であるためです。

C++20 以降（Concepts 機能）では `requires` を変数名として使用できなくなりました。そのため、コンパイラが変数宣言ではなく構文エラー（

expected unqualified-id) として処理してしまっています。

変数名を req_val や reqFlags などに変更することで即座に解決します。

修正コード

C++



```
for (const auto& item : containerB) {
    uint64_t prop = item.prop;

    unsigned root      = (prop >> 16) & 0xFF; // 8ビット幅 (newRoot)
    unsigned hasRead    = (prop >> 34) & 0x1;  // 1ビット
    unsigned reqFlags   = (prop >> 48) & 0xFF; // requires から reqFlags に
    unsigned excFlags   = (prop >> 56) & 0xFF; // excludes も念のため excFl

    // 条件判定
    bool cond1 = (root == gRoot);
    bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
    bool cond3 = (hasRead == 0);
    bool cond4 = ((gUnlocks & excFlags) == 0);

    if (cond1 && cond2 && cond3 && cond4) {
        containerC.push_back(item);
    }
}
```

コンテナCのtextを画面に最大4つ表示する。コンテナCに5つ以上要素がある場合は、ランダムに4つを選び、画面に4つ表示する。

ただし、コンテナCに一つでもisOnetime && isQl



表示要素のランダム選出と、選択中の要素を管理するインデックス（カーソル）、およびキー入力イベント（'i' / 'o'）による表示色の切り替え処理を実装します。

C++の `<random>` を使用してランダム選択を行い、SDLのイベントループ内でキー入力を監視して選択位置を更新します。

変更点と実装のポイント

1. `isOnetime` と `isQuote` の判定:

`isOnetime [33]: (prop >> 33) & 1`

`isQuote [32]: (prop >> 32) & 1`

2. 表示用リストの作成:

条件を満たす要素（`isOnetime && isQuote`）が含まれる場合 → ランダムに1つ選出。

条件を満たさず、要素が5つ以上の場合 → ランダムに4つ選出（`std::shuffle` を使用）。

要素が4つ以下の場合はそのまますべて使用。

3. キー入力とインデックス循環:

'o' (右移動):

`selectedIndex = (selectedIndex + 1) % displayList.size()`

'i' (左移動):

`selectedIndex = (selectedIndex - 1 + displayList.size()) % displayList.size()`

完全なプログラムコード

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>
```

```
struct Element {
```

```

    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;

// 画面表示用の要素リスト
std::vector<Element> displayList;

// 選択中の要素インデックス（デフォルト: 0 = 一番左）
size_t selectedIndex = 0;

// グローバルな条件判定用変数
unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gUnlocks = 0;

// ANSIエスケープコードによる表示更新関数
void renderConsole() {
    // 画面クリア（コンソール用）
    std::cout << "\033[2J\033[11H";

    if (displayList.empty()) {
        std::cout << "no content" << std::endl;
        return;
    }

    // 表示処理
    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            // 赤色表示 (\033[31m)
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        } else {
            // 白色表示 (\033[37m)
            std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
        }
    }
    std::cout << std::endl;
    std::cout << "(Press 'o' to move right, 'i' to move left, Esc/Quit to ex
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    // -----
    // 1. ファイル読み込み (dd.txt -> containerA)
    // -----
    const char *base_path = SDL_GetBasePath();

```

```

std::string file_path = base_path ? std::string(base_path) + "dd.txt" :

std::ifstream file(file_path);
if (!file.is_open()) {
    std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
    return SDL_APP_FAILURE;
}

std::string token;
std::string current_text;
bool is_text = true;

while (file >> token) {
    std::string clean_token = "";
    for (char c : token) {
        if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
            clean_token += c;
    }
    if (clean_token.empty()) continue;

    if (is_text) {
        current_text = clean_token;
        is_text = false;
    } else {
        uint64_t prop = std::stoull(clean_token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}
file.close();

// -----
// 2. containerA -> containerB 抽出
// -----
for (const auto& item : containerA) {
    unsigned scene = (item.prop >> 36) & 0xF;
    if (scene == gScene) {
        containerB.push_back(item);
    }
}

// -----
// 3. containerB -> containerC 抽出
// -----
for (const auto& item : containerB) {
    uint64_t prop = item.prop;

    unsigned root      = (prop >> 16) & 0xFF;

```

```

unsigned hasRead  = (prop >> 34) & 0x1;
unsigned reqFlags = (prop >> 48) & 0xFF;
unsigned excFlags = (prop >> 56) & 0xFF;

bool cond1 = (root == gRoot);
bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
bool cond3 = (hasRead == 0);
bool cond4 = ((gUnlocks & excFlags) == 0);

if (cond1 && cond2 && cond3 && cond4) {
    containerC.push_back(item);
}
}

// -----
// 4. コンテナCから displayList への選出処理
// -----
if (!containerC.empty()) {
    std::random_device rd;
    std::mt19937 gen(rd());

    // 特別条件: isOnetime [33] && isQuote [32] == true の要素が存在するか?
    bool hasOnetimeQuote = false;
    for (const auto& item : containerC) {
        bool isOnetime = (item.prop >> 33) & 0x1;
        bool isQuote    = (item.prop >> 32) & 0x1;
        if (isOnetime && isQuote) {
            hasOnetimeQuote = true;
            break;
        }
    }

    if (hasOnetimeQuote) {
        // 条件を満たす要素が含まれる場合はランダムに1つだけ選出
        std::uniform_int_distribution<size_t> dist(0, containerC.size()
            displayList.push_back(containerC[dist(gen)]));
    } else if (containerC.size() >= 5) {
        // 5つ以上ある場合はシャッフルして先頭4つを抽出
        std::vector<Element> temp = containerC;
        std::shuffle(temp.begin(), temp.end(), gen);
        displayList.assign(temp.begin(), temp.begin() + 4);
    } else {
        // 4つ以下の場合はそのまま使用
        displayList = containerC;
    }
}

// 初期描画
renderConsole();

```

```

        return SDL_APP_SUCCESS;
    }

// -----
// 5. イベント処理 ('i' / 'o' のキー入力判定)
// -----
SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        if (!displayList.empty()) {
            size_t total = displayList.size();

            if (event->key.key == SDLK_0) {
                // 右移動 (一番右の場合は一番左へ)
                selectedIndex = (selectedIndex + 1) % total;
                renderConsole();
            } else if (event->key.key == SDLK_I) {
                // 左移動 (一番左の場合は一番右へ)
                selectedIndex = (selectedIndex - 1 + total) % total;
                renderConsole();
            }
        }
    }

    return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppIterate(void *appstate) {
    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
}

```

コンソールではなく SDL_Rendererを使ったグラフィック描画に変更しますか？

⇒ はい

実行したら、すぐプログラムが終了しちゃう。
BUILD_TYPE=Debugにしているのが原因かな。

BUILD_TYPE=Debug が直接の原因ではありません。

すぐに終了してしまう理由は、SDL_AppInit の最後で
return SDL_APP_SUCCESS; を返しているためです。

SDL3 (SDL_MAIN_USE_CALLBACKS) では、コールバック関数の戻り値によって
アプリケーションの状態を制御します。

SDL_APP_CONTINUE : アプリケーションを継続する (イベント待ち・
描画ループへ入る)

SDL_APP_SUCCESS : 正常終了する (即座に SDL_AppQuit が呼ばれて終了)

SDL_APP_FAILURE : エラー終了する

解決方法

SDL_AppInit の最後の戻り値を SDL_APP_CONTINUE に変更してください。

C++



```
SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {  
    // ... 前半のファイル読み込みやコンテナ処理 ...  
  
    // 初期描画  
    renderConsole();  
  
    // 変更前: return SDL_APP_SUCCESS; (これで即終了していた)  
    // 変更後: ループ処理・イベント受け取りを継続するために CONTINUE を返す  
    return SDL_APP_CONTINUE;  
}
```

イベントループを維持するためのポイント

また、SDL_AppIterate 内でも継続してアプリを動かし付けるために
SDL_APP_CONTINUE を返しておく必要があります。

C++



```
SDL_AppResult SDL_AppIterate(void *appstate) {
    // 画面更新やメインループを継続する
    return SDL_APP_CONTINUE;
}
```

これで `SDL_AppInit` 完了後もプログラムが終了せず待機状態になり、`'i'` や `'o'` のキー入力を正しく受け付けられるようになります。

```
描画が更新されない。[konichiwa] [ohayo] [arigato]
(Press 'o' to move right, 'i' to move left, Esc/Quit to exit)
o
oooo
```

キー入力で描画が更新されず、入力した文字（`o` や `i`）がそのままコンソールにエコーバック表示されてしまう理由は主に2つあります。

1. **GUIウィンドウが生成されていない:** SDLはSDLが生成したウィンドウがフォーカスを持っている時のみキーボードイベント（`SDL_EVENT_KEY_DOWN`）を検知します。端末（ターミナル）上でキーを押してもSDLイベントは発火しません。
2. **コンソールの標準出力バッファ:** `std::cout` に `std::flush`（または `std::endl`）が正しく送られていない場合、再描画処理がターミナルに即座に反映されません。

修正コード

SDLウィンドウを1つ生成してイベントを受け取れる状態にし、`q` キーでの終了処理を追加しました。

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>

struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;
std::vector<Element> displayList;

size_t selectedIndex = 0;
SDL_Window *gWindow = nullptr; // イベント送信用ウィンドウ

unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gUnlocks = 0;

void renderConsole() {
    // 画面クリア (ANSIエスケープシーケンス)
    std::cout << "\033[2J\033[1;1H";

    if (displayList.empty()) {
        std::cout << "no content" << std::endl;
        return;
    }

    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            // 赤色表示
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        } else {
            // 白色表示
            std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
        }
    }
}
```

```

    }
    std::cout << std::endl;
    std::cout << "(Focus SDL Window -> 'o': Right, 'i': Left, 'q': Quit)" <<
    std::cout << std::flush; // 出力を即座にコンソールへ反映
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    // キーイベントを受け取るためのダミーウィンドウを作成
    gWindow = SDL_CreateWindow("DDMEditor Key Input Focus", 320, 240, 0);
    if (!gWindow) {
        std::cerr << "Window Creation Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    // -----
    // 1. ファイル読み込み
    // -----
    const char *base_path = SDL_GetBasePath();
    std::string file_path = base_path ? std::string(base_path) + "dd.txt" :

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::endl;
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;
    bool is_text = true;

    while (file >> token) {
        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }

        if (clean_token.empty()) continue;

        if (is_text) {
            current_text = clean_token;
            is_text = false;
        } else {
            uint64_t prop = std::stoull(clean_token, nullptr, 16);

```

```

        containerA.push_back({current_text, prop});
        is_text = true;
    }
}
file.close();

// -----
// 2. フィルタリング (A -> B -> C)
// -----
for (const auto& item : containerA) {
    unsigned scene = (item.prop >> 36) & 0xF;
    if (scene == gScene) {
        containerB.push_back(item);
    }
}

for (const auto& item : containerB) {
    uint64_t prop = item.prop;

    unsigned root      = (prop >> 16) & 0xFF;
    unsigned hasRead   = (prop >> 34) & 0x1;
    unsigned reqFlags  = (prop >> 48) & 0xFF;
    unsigned excFlags  = (prop >> 56) & 0xFF;

    bool cond1 = (root == gRoot);
    bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
    bool cond3 = (hasRead == 0);
    bool cond4 = ((gUnlocks & excFlags) == 0);

    if (cond1 && cond2 && cond3 && cond4) {
        containerC.push_back(item);
    }
}

// -----
// 3. ランダム抽出
// -----
if (!containerC.empty()) {
    std::random_device rd;
    std::mt19937 gen(rd());

    bool hasOnetimeQuote = false;
    for (const auto& item : containerC) {
        bool isOnetime = (item.prop >> 33) & 0x1;
        bool isQuote   = (item.prop >> 32) & 0x1;
        if (isOnetime && isQuote) {
            hasOnetimeQuote = true;
            break;
        }
    }
}

```

```

    }

    if (hasOnetimeQuote) {
        std::uniform_int_distribution<size_t> dist(0, containerC.size());
        displayList.push_back(containerC[dist(gen)]);
    } else if (containerC.size() >= 5) {
        std::vector<Element> temp = containerC;
        std::shuffle(temp.begin(), temp.end(), gen);
        displayList.assign(temp.begin(), temp.begin() + 4);
    } else {
        displayList = containerC;
    }
}

renderConsole();
return SDL_APP_CONTINUE;
}

// -----
// 4. イベント処理 ('i', 'o', 'q')
// -----
SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        // 'q' キーで正常終了
        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (!displayList.empty()) {
            size_t total = displayList.size();

            if (key == SDLK_0) {
                selectedIndex = (selectedIndex + 1) % total;
                renderConsole();
            } else if (key == SDLK_I) {
                selectedIndex = (selectedIndex - 1 + total) % total;
                renderConsole();
            }
        }
    }

    return SDL_APP_CONTINUE;
}

```

```

SDL_AppResult SDL_AppIterate(void *appstate) {
    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    if (gWindow) {
        SDL_DestroyWindow(gWindow);
        gWindow = nullptr;
    }
}

```

操作手順

実行すると小ウィンドウ（DDMEditor Key Input Focus）が表示されます。このウィンドウをクリックしてアクティブ（フォーカス）にした状態で 'o' や 'i'、'q' を押すと、ターミナル側の色が正しく切り替わり、'q' で正常終了します。

ウィンドウは開かないよ。AppIterateの中に書かないとだめなんじゃない？

仰る通りです！SDL3のコールバック機構（SDL_MAIN_USE_CALLBACKS）では、SDL_AppInit でウィンドウを作成した後、SDL_AppIterate 内で描画フレームを更新・レンダリングしないと、OS側でウィンドウが正常に表示・更新されません。

また、コンソール出力を行う場合でも SDL_AppIterate 内で再描画フラグを監視して更新をかけることで、キー入力と連動してスムーズに表示が切り替わるようになります。

修正のポイント

1. SDL_AppIterate 内での描画駆動
キーイベントで「再描画が必要（needRedraw = true）」のフラグを立て、SDL_AppIterate のタイミングで renderConsole()（およびウィンドウのレンダリング）を呼び出します。
2. ウィンドウ背景のクリア
SDL_AppIterate 内で SDL_RenderClear と SDL_RenderPresent を呼ぶことで、OS側に「ウィンドウが活着ている」ことを正しく通知します。

完全な修正コード

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>

struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;
std::vector<Element> displayList;

size_t selectedIndex = 0;
bool needRedraw = true; // 再描画フラグ

SDL_Window *gWindow = nullptr;
SDL_Renderer *gRenderer = nullptr;

unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gUnlocks = 0;

void renderConsole() {
    std::cout << "\033[2J\033[1;1H"; // 画面クリア

    if (displayList.empty()) {
        std::cout << "no content" << std::endl;
        return;
    }

    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        }
    }
}
```

```

    } else {
        std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
    }
}

std::cout << std::endl;
std::cout << "(Focus SDL Window -> 'o': Right, 'i': Left, 'q': Quit)" <<
std::cout << std::flush;
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    // ウィンドウとレンダラーの生成
    if (!SDL_CreateWindowAndRenderer("DDMEditor Focus Window", 320, 240, 0,
        std::cerr << "Window/Renderer Creation Error: " << SDL_GetError() <<
        return SDL_APP_FAILURE;
    }

    // -----
    // 1. ファイル読み込み (dd.txt -> containerA)
    // -----
    const char *base_path = SDL_GetBasePath();
    std::string file_path = base_path ? std::string(base_path) + "dd.txt" :

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;
    bool is_text = true;

    while (file >> token) {
        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }
        if (clean_token.empty()) continue;

        if (is_text) {
            current_text = clean_token;
            is_text = false;

```

```

    } else {
        uint64_t prop = std::stoull(clean_token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}

file.close();

// -----
// 2. フィルタリング (A -> B -> C)
// -----
for (const auto& item : containerA) {
    unsigned scene = (item.prop >> 36) & 0xF;
    if (scene == gScene) {
        containerB.push_back(item);
    }
}

for (const auto& item : containerB) {
    uint64_t prop = item.prop;

    unsigned root      = (prop >> 16) & 0xFF;
    unsigned hasRead   = (prop >> 34) & 0x1;
    unsigned reqFlags  = (prop >> 48) & 0xFF;
    unsigned excFlags  = (prop >> 56) & 0xFF;

    bool cond1 = (root == gRoot);
    bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
    bool cond3 = (hasRead == 0);
    bool cond4 = ((gUnlocks & excFlags) == 0);

    if (cond1 && cond2 && cond3 && cond4) {
        containerC.push_back(item);
    }
}

// -----
// 3. ランダム抽出
// -----
if (!containerC.empty()) {
    std::random_device rd;
    std::mt19937 gen(rd());

    bool hasOnetimeQuote = false;
    for (const auto& item : containerC) {
        bool isOnetime = (item.prop >> 33) & 0x1;
        bool isQuote   = (item.prop >> 32) & 0x1;
        if (isOnetime && isQuote) {
            hasOnetimeQuote = true;
        }
    }
}

```

```

        break;
    }
}

if (hasOneTimeQuote) {
    std::uniform_int_distribution<size_t> dist(0, containerC.size());
    displayList.push_back(containerC[dist(gen)]);
} else if (containerC.size() >= 5) {
    std::vector<Element> temp = containerC;
    std::shuffle(temp.begin(), temp.end(), gen);
    displayList.assign(temp.begin(), temp.begin() + 4);
} else {
    displayList = containerC;
}

return SDL_APP_CONTINUE;
}

// -----
// 4. イベント処理
// -----
SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (!displayList.empty()) {
            size_t total = displayList.size();

            if (key == SDLK_0) {
                selectedIndex = (selectedIndex + 1) % total;
                needRedraw = true; // 再描画要求
            } else if (key == SDLK_I) {
                selectedIndex = (selectedIndex - 1 + total) % total;
                needRedraw = true; // 再描画要求
            }
        }
    }

    return SDL_APP_CONTINUE;
}

```

```
// -----
// 5. 毎フレームの更新・レンダリング（重要）
// -----
SDL_AppResult SDL_AppIterate(void *appstate) {
    // フラグがある時だけコンソールを再描画
    if (needRedraw) {
        renderConsole();
        needRedraw = false;
    }

    // ウィンドウ画面の描画更新（OSにウィンドウが正常応答していることを伝える）
    SDL_SetRenderDrawColor(gRenderer, 30, 30, 30, 255);
    SDL_RenderClear(gRenderer);
    SDL_RenderPresent(gRenderer);

    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    if (gRenderer) {
        SDL_DestroyRenderer(gRenderer);
        gRenderer = nullptr;
    }
    if (gWindow) {
        SDL_DestroyWindow(gWindow);
        gWindow = nullptr;
    }
}
```

'f'を入力するとpropertyを送信する。
 送信したpropertyに対して、以下のような操作を行う。
 newScene != 0なら、gScene = newScene, gRoot = 0,
 gUnlocks=0として、containerB, Cを初期化し、ふたつの
 コンテナを上の手順と同じように再構築する。送信処理を
 完了する。
 gUnlocks |= unlocksのようにして、unlocksの通りフラグを
 立てる。
 isOnetime==trueなら、hasReadフラグを立てる。
 newRoot !=0なら、root = newRootとする。
 以上で送信処理を完了し、containerCを再構築する。
 再び、containerCの内容を上ルールで配信する。

へ

'f' キー入力時に、現在選択されている要素（`displayList[selectedIndex]`）の `prop` から各ビットフィールド（`newScene`, `unlocks`, `isOnetime`, `newRoot` 等）を取り出し、指定の順序でグローバル変数の更新、`hasRead` フラグの更新、およびコンテナの再構築を行うロジックを追加します。

更新ロジックのフロー

1. 選択中プロパティの抽出:

`newScene [24-31]: (prop >> 24) & 0xFF`

`unlocks [40-47]: (prop >> 40) & 0xFF`

`isOnetime [33]: (prop >> 33) & 1`

`newRoot [16-23]: (prop >> 16) & 0xFF`

2. `newScene != 0` の場合:

`gScene = newScene`, `gRoot = 0`, `gUnlocks = 0` に更新。

`containerA` 内の該当要素の `hasRead` フラグ（34ビット目）も更新:
`isOnetime == true` の場合は `hasRead` ビットを 1 に書き換えます (`prop |= (1ULL << 34)`)。

`containerB`, `containerC` を破棄して再構築し、`displayList` を選出して処理完了。

3. `newScene == 0` の場合:

`gUnlocks |= unlocks` でフラグを更新。

`isOnetime == true` なら、`containerA` の元の要素の `hasRead` フラグ（34ビット目）を立てます。

`newRoot != 0` なら `gRoot = newRoot` とします。

`containerC` を再構築し、`displayList` を再選出して処理完了。

完全なプログラムコード

```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>

struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;
std::vector<Element> displayList;

size_t selectedIndex = 0;
bool needRedraw = true;

SDL_Window *gWindow = nullptr;
SDL_Renderer *gRenderer = nullptr;

unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gUnlocks = 0;

// コンテナ構築・抽出関数プロトタイプ
void rebuildContainerB();
void rebuildContainerC();
void rebuildDisplayList();

void renderConsole() {
    std::cout << "\033[2J\033[1;1H";

    std::cout << "[ State ] gScene: " << gScene
                << ", gRoot: " << gRoot
                << ", gUnlocks: 0x" << std::hex << gUnlocks << std::dec << "\r\n";

    if (displayList.empty()) {
        std::cout << "no content" << std::endl;
        return;
    }
}
```

```

    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        } else {
            std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
        }
    }
    std::cout << std::endl;
    std::cout << "(Focus Window -> 'o': Right, 'i': Left, 'f': Send, 'q': Quit) ";
    std::cout << std::flush;
}

// -----
// コンテナ再構築ロジック群
// -----
void rebuildContainerB() {
    containerB.clear();
    for (const auto& item : containerA) {
        unsigned scene = (item.prop >> 36) & 0xF;
        if (scene == gScene) {
            containerB.push_back(item);
        }
    }
}

void rebuildContainerC() {
    containerC.clear();
    for (const auto& item : containerB) {
        uint64_t prop = item.prop;

        unsigned root      = (prop >> 16) & 0xFF;
        unsigned hasRead    = (prop >> 34) & 0x1;
        unsigned reqFlags   = (prop >> 48) & 0xFF;
        unsigned excFlags   = (prop >> 56) & 0xFF;

        bool cond1 = (root == gRoot);
        bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
        bool cond3 = (hasRead == 0);
        bool cond4 = ((gUnlocks & excFlags) == 0);

        if (cond1 && cond2 && cond3 && cond4) {
            containerC.push_back(item);
        }
    }
}

void rebuildDisplayList() {
    displayList.clear();
    selectedIndex = 0;
}

```

```

    if (!containerC.empty()) {
        std::random_device rd;
        std::mt19937 gen(rd());

        bool hasOnetimeQuote = false;
        for (const auto& item : containerC) {
            bool isOnetime = (item.prop >> 33) & 0x1;
            bool isQuote    = (item.prop >> 32) & 0x1;
            if (isOnetime && isQuote) {
                hasOnetimeQuote = true;
                break;
            }
        }

        if (hasOnetimeQuote) {
            std::uniform_int_distribution<size_t> dist(0, containerC.size()
            displayList.push_back(containerC[dist(gen)]);
        } else if (containerC.size() >= 5) {
            std::vector<Element> temp = containerC;
            std::shuffle(temp.begin(), temp.end(), gen);
            displayList.assign(temp.begin(), temp.begin() + 4);
        } else {
            displayList = containerC;
        }
    }
}

// -----
// 'f' キー入力時の送信処理
// -----
void sendProperty() {
    if (displayList.empty() || selectedIndex >= displayList.size()) return;

    Element selected = displayList[selectedIndex];
    uint64_t prop = selected.prop;

    unsigned newScene  = (prop >> 24) & 0xFF;
    unsigned unlocks   = (prop >> 40) & 0xFF;
    bool      isOnetime = (prop >> 33) & 0x1;
    unsigned newRoot    = (prop >> 16) & 0xFF;

    // containerA 内の該当要素を探して isOnetime フラグ処理 (hasRead ビット=34 を立
    if (isOnetime) {
        for (auto& item : containerA) {
            if (item.text == selected.text && item.prop == selected.prop) {
                item.prop |= (1ULL << 34); // hasRead (bit 34) を 1 に変更
                break;
            }
        }
    }
}

```

```

    }
}

if (newScene != 0) {
    // newScene != 0 の場合: gScene変更・コンテナB, Cを全再構築
    gScene = newScene;
    gRoot = 0;
    gUnlocks = 0;

    rebuildContainerB();
    rebuildContainerC();
    rebuildDisplayList();
} else {
    // newScene == 0 の場合
    gUnlocks |= unlocks;

    if (newRoot != 0) {
        gRoot = newRoot;
    }

    // isOnetime によって hasRead が書き換わった状態をコンテナBにも反映後、Cを再構築
    rebuildContainerB();
    rebuildContainerC();
    rebuildDisplayList();
}

needRedraw = true;
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    if (!SDL_CreateWindowAndRenderer("DDMEditor Focus Window", 320, 240, 0,
        std::cerr << "Window/Renderer Creation Error: " << SDL_GetError() <<
        return SDL_APP_FAILURE;
    }

    // 1. ファイル読み込み
    const char *base_path = SDL_GetBasePath();
    std::string file_path = base_path ? std::string(base_path) + "dd.txt" :

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
        return SDL_APP_FAILURE;
    }
}

```

```

std::string token;
std::string current_text;
bool is_text = true;

while (file >> token) {
    std::string clean_token = "";
    for (char c : token) {
        if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
            clean_token += c;
    }
    if (clean_token.empty()) continue;

    if (is_text) {
        current_text = clean_token;
        is_text = false;
    } else {
        uint64_t prop = std::stoull(clean_token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}
file.close();

// 初回の全コンテナ構築
rebuildContainerB();
rebuildContainerC();
rebuildDisplayList();

return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (key == SDLK_F) {
            sendProperty();
        } else if (!displayList.empty()) {
            size_t total = displayList.size();

```

```

        if (key == SDLK_O) {
            selectedIndex = (selectedIndex + 1) % total;
            needRedraw = true;
        } else if (key == SDLK_I) {
            selectedIndex = (selectedIndex - 1 + total) % total;
            needRedraw = true;
        }
    }

    return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppIterate(void *appstate) {
    if (needRedraw) {
        renderConsole();
        needRedraw = false;
    }

    SDL_SetRenderDrawColor(gRenderer, 30, 30, 30, 255);
    SDL_RenderClear(gRenderer);
    SDL_RenderPresent(gRenderer);

    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    if (gRenderer) {
        SDL_DestroyRenderer(gRenderer);
        gRenderer = nullptr;
    }
    if (gWindow) {
        SDL_DestroyWindow(gWindow);
        gWindow = nullptr;
    }
}

```

デバッグ用のdd.txtを出力して。たとえば、parent=0で'見る','話す','行く'のtextを設定し、それらのidを親として、a氏、公園のように、動詞と目的語のtextを採用するとよい。ただし、コメントがあれば、'と,の間にか' ✓

指示された内容に沿って修正を行ったプログラムと、デバッグ用の dd.txt の出力フォーマットです。

1. デバッグ用 dd.txt

ファイル位置は main.cpp （またはビルドフォルダ）から見て1つ上の階層（`../dd.txt`）に配置してください。

コメントは `・`（シングルクォート）の外側に記述しているため、パース時に無視され containerA の構築に影響しません。

Plaintext



```
/* === Parent = 0（動詞） === */
// [0] 見る (scene=0, root=0, id=1, newRoot=1, unlocks=0x01)
'見る', '0x0000000001010001',

// [1] 話す (scene=0, root=0, id=2, newRoot=2, unlocks=0x02)
'話す', '0x0000000002020002',

// [2] 行く (scene=0, root=0, id=3, newRoot=3, unlocks=0x04)
'行く', '0x0000000004030003',

/* === Parent = 1（「見る」の目的語） === */
// [3] a氏 (scene=0, parent=1, id=4, requires=0x01)
'a氏', '0x0001000000000104',

// [4] 公園 (scene=0, parent=1, id=5, requires=0x01)
'公園', '0x0001000000000105',

/* === Parent = 2（「話す」の目的語） === */
// [5] a氏 (scene=0, parent=2, id=6, requires=0x02)
'a氏', '0x0002000000000206',

/* === Parent = 3（「行く」の目的語） === */
// [6] 公園 (scene=0, parent=3, id=7, requires=0x04)
'公園', '0x0004000000000307'
```

2. 修正版プログラム (main.cpp)

主な変更点：

1. パスの固定: file_path を常に `"../dd.txt"` に固定。
2. cond1 の条件変更: root (newRoot) ではなく parent ([8-15]) と gRoot を比較するよう修正 (

```
    unsigned parent = (prop >> 8) & 0xFF; bool cond1 = (parent == gRoot);  
    )。
```

C++



```
#define SDL_MAIN_USE_CALLBACKS 1  
#include <SDL3/SDL.h>  
#include <SDL3/SDL_main.h>  
#include <iostream>  
#include <fstream>  
#include <string>  
#include <vector>  
#include <cstdint>  
#include <random>  
#include <algorithm>  
  
struct Element {  
    std::string text;  
    uint64_t prop;  
};  
  
// グローバル変数  
std::vector<Element> containerA;  
std::vector<Element> containerB;  
std::vector<Element> containerC;  
std::vector<Element> displayList;  
  
size_t selectedIndex = 0;  
bool needRedraw = true;  
  
SDL_Window *gWindow = nullptr;  
SDL_Renderer *gRenderer = nullptr;  
  
unsigned gScene = 0;  
unsigned gRoot = 0;  
unsigned gUnlocks = 0;  
  
void rebuildContainerB();  
void rebuildContainerC();  
void rebuildDisplayList();  
  
void renderConsole() {  
    std::cout << "\033[2J\033[1;1H";  
  
    std::cout << "[ State ] gScene: " << gScene  
                << ", gRoot: " << gRoot
```

```
<< ", gUnlocks: 0x" << std::hex << gUnlocks << std::dec << "\r
```

```

if (displayList.empty()) {
    std::cout << "no content" << std::endl;
    return;
}

for (size_t i = 0; i < displayList.size(); ++i) {
    if (i == selectedIndex) {
        std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
    } else {
        std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
    }
}

std::cout << std::endl;
std::cout << "(Focus Window -> 'o': Right, 'i': Left, 'f': Send, 'q': Q
std::cout << std::flush;
}

void rebuildContainerB() {
    containerB.clear();
    for (const auto& item : containerA) {
        unsigned scene = (item.prop >> 36) & 0xF;
        if (scene == gScene) {
            containerB.push_back(item);
        }
    }
}

void rebuildContainerC() {
    containerC.clear();
    for (const auto& item : containerB) {
        uint64_t prop = item.prop;

        // cond1 の判定対象を root (16-23) から parent (8-15) に変更
        unsigned parent    = (prop >> 8) & 0xFF;
        unsigned hasRead    = (prop >> 34) & 0x1;
        unsigned reqFlags   = (prop >> 48) & 0xFF;
        unsigned excFlags   = (prop >> 56) & 0xFF;

        bool cond1 = (parent == gRoot); // parent と gRoot を比較
        bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
        bool cond3 = (hasRead == 0);
        bool cond4 = ((gUnlocks & excFlags) == 0);

        if (cond1 && cond2 && cond3 && cond4) {
            containerC.push_back(item);
        }
    }
}

```

```

}
}

```

```

void rebuildDisplayList() {
    displayList.clear();
    selectedIndex = 0;

    if (!containerC.empty()) {
        std::random_device rd;
        std::mt19937 gen(rd());

        bool hasOnetimeQuote = false;
        for (const auto& item : containerC) {
            bool isOnetime = (item.prop >> 33) & 0x1;
            bool isQuote    = (item.prop >> 32) & 0x1;
            if (isOnetime && isQuote) {
                hasOnetimeQuote = true;
                break;
            }
        }

        if (hasOnetimeQuote) {
            std::uniform_int_distribution<size_t> dist(0, containerC.size()
            displayList.push_back(containerC[dist(gen)]);
        } else if (containerC.size() >= 5) {
            std::vector<Element> temp = containerC;
            std::shuffle(temp.begin(), temp.end(), gen);
            displayList.assign(temp.begin(), temp.begin() + 4);
        } else {
            displayList = containerC;
        }
    }
}

```

```

void sendProperty() {
    if (displayList.empty() || selectedIndex >= displayList.size()) return;

    Element selected = displayList[selectedIndex];
    uint64_t prop = selected.prop;

    unsigned newScene  = (prop >> 24) & 0xFF;
    unsigned unlocks   = (prop >> 40) & 0xFF;
    bool        isOnetime = (prop >> 33) & 0x1;
    unsigned newRoot    = (prop >> 16) & 0xFF;

    if (isOnetime) {
        for (auto& item : containerA) {
            if (item.text == selected.text && item.prop == selected.prop) {
                item.prop |= (1ULL << 34);
                break;
            }
        }
    }
}

```

```

    }
    }
}

if (newScene != 0) {
    gScene = newScene;
    gRoot = 0;
    gUnlocks = 0;

    rebuildContainerB();
    rebuildContainerC();
    rebuildDisplayList();
} else {
    gUnlocks |= unlocks;

    if (newRoot != 0) {
        gRoot = newRoot;
    }

    rebuildContainerB();
    rebuildContainerC();
    rebuildDisplayList();
}

needRedraw = true;
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    if (!SDL_CreateWindowAndRenderer("DDMEditor Focus Window", 320, 240, 0,
        std::cerr << "Window/Renderer Creation Error: " << SDL_GetError() <<
        return SDL_APP_FAILURE;
    }

    // パスを常に "../dd.txt" に固定
    std::string file_path = "../dd.txt";

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::endl;
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;

```

```

    bool is_text = true;

    while (file >> token) {
        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }
        if (clean_token.empty()) continue;

        if (is_text) {
            current_text = clean_token;
            is_text = false;
        } else {
            uint64_t prop = std::stoull(clean_token, nullptr, 16);
            containerA.push_back({current_text, prop});
            is_text = true;
        }
    }
    file.close();

    rebuildContainerB();
    rebuildContainerC();
    rebuildDisplayList();

    return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (key == SDLK_F) {
            sendProperty();
        } else if (!displayList.empty()) {
            size_t total = displayList.size();

            if (key == SDLK_O) {
                selectedIndex = (selectedIndex + 1) % total;
                needRedraw = true;
            }
        }
    }
}

```

```

    } else if (key == SDLK_I) {
        selectedIndex = (selectedIndex - 1 + total) % total;
        needRedraw = true;
    }
}

return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppIterate(void *appstate) {
    if (needRedraw) {
        renderConsole();
        needRedraw = false;
    }

    SDL_SetRenderDrawColor(gRenderer, 30, 30, 30, 255);
    SDL_RenderClear(gRenderer);
    SDL_RenderPresent(gRenderer);

    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    if (gRenderer) {
        SDL_DestroyRenderer(gRenderer);
        gRenderer = nullptr;
    }
    if (gWindow) {
        SDL_DestroyWindow(gWindow);
        gWindow = nullptr;
    }
}

```

送信処理に以下の内容を追加し、修正する。
 globalのunsigned gParent = idとする。
 cond 1をgParent == parentに修正し、containerCを
 再構築する。

✓

sendProperty() 内での gParent の更新処理、rebuildContainerC() における
 条件判定の変更、および要素数 0 時のフォールバック & no content 一時
 表示処理を追加・修正したコードです。

修正内容のポイント

1. `gParent` 変数の追加: グローバル変数に `unsigned gParent = 0;` を定義し、`sendProperty()` 時に選択された要素の `id` (ビット 0-7) を `gParent` に代入。
2. `cond1` の条件修正: `rebuildContainerC()` 内で `bool cond1 = (parent == gParent);` に変更。
3. 要素数 0 時の再構築と `no content` 表示:

`containerC` が空 (`empty()`) の場合、`gParent = gRoot;` に書き換えて再度 `rebuildContainerC()` を実行。

`gParent = gRoot` によるフォールバックが発生した際、画面描画 (`renderConsole()`) を挟んで "no content" を一時表示した上で、改めて `displayList` を構築して最終画面を描画。

修正版プログラム (main.cpp)

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>

struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;
std::vector<Element> displayList;

size_t selectedIndex = 0;
bool needRedraw = true;
```

```

SDL_Window *gWindow = nullptr;
SDL_Renderer *gRenderer = nullptr;

unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gParent = 0; // 追加
unsigned gUnlocks = 0;

void rebuildContainerB();
void rebuildContainerC();
void rebuildDisplayList();

void renderConsole() {
    std::cout << "\033[2J\033[1;1H";

    std::cout << "[ State ] gScene: " << gScene
                << ", gRoot: " << gRoot
                << ", gParent: " << gParent
                << ", gUnlocks: 0x" << std::hex << gUnlocks << std::dec << "\r\n";

    if (displayList.empty()) {
        std::cout << "no content" << std::endl;
        return;
    }

    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        } else {
            std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
        }
    }

    std::cout << std::endl;
    std::cout << "(Focus Window -> 'o': Right, 'i': Left, 'f': Send, 'q': Quit)";
    std::cout << std::flush;
}

void rebuildContainerB() {
    containerB.clear();
    for (const auto& item : containerA) {
        unsigned scene = (item.prop >> 36) & 0xF;
        if (scene == gScene) {
            containerB.push_back(item);
        }
    }
}

void rebuildContainerC() {

```

```

    containerC.clear();
    for (const auto& item : containerB) {
        uint64_t prop = item.prop;

        unsigned parent    = (prop >> 8) & 0xFF;
        unsigned hasRead    = (prop >> 34) & 0x1;
        unsigned reqFlags   = (prop >> 48) & 0xFF;
        unsigned excFlags   = (prop >> 56) & 0xFF;

        // cond1 の判定対象を parent == gParent に修正
        bool cond1 = (parent == gParent);
        bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
        bool cond3 = (hasRead == 0);
        bool cond4 = ((gUnlocks & excFlags) == 0);

        if (cond1 && cond2 && cond3 && cond4) {
            containerC.push_back(item);
        }
    }
}

void rebuildDisplayList() {
    displayList.clear();
    selectedIndex = 0;

    if (!containerC.empty()) {
        std::random_device rd;
        std::mt19937 gen(rd());

        bool hasOnetimeQuote = false;
        for (const auto& item : containerC) {
            bool isOnetime = (item.prop >> 33) & 0x1;
            bool isQuote    = (item.prop >> 32) & 0x1;
            if (isOnetime && isQuote) {
                hasOnetimeQuote = true;
                break;
            }
        }

        if (hasOnetimeQuote) {
            std::uniform_int_distribution<size_t> dist(0, containerC.size()
            displayList.push_back(containerC[dist(gen)]);
        } else if (containerC.size() >= 5) {
            std::vector<Element> temp = containerC;
            std::shuffle(temp.begin(), temp.end(), gen);
            displayList.assign(temp.begin(), temp.begin() + 4);
        } else {
            displayList = containerC;
        }
    }
}

```

```

    }
}

void processContainerLifecycle() {
    rebuildContainerB();
    rebuildContainerC();

    // 取得結果が0件の場合の再構築フロー
    if (containerC.empty()) {
        gParent = gRoot;
        rebuildContainerC();

        // 1度 no content を表示するために displayList を空にして描画
        displayList.clear();
        renderConsole();

        // SDLのイベントループを阻害しないようミリ秒単位で軽く視認用の待機を入れる
        SDL_Delay(300);

        rebuildDisplayList();
    } else {
        rebuildDisplayList();
    }
}

void sendProperty() {
    if (displayList.empty() || selectedIndex >= displayList.size()) return;

    Element selected = displayList[selectedIndex];
    uint64_t prop = selected.prop;

    unsigned id          = prop & 0xFF; // ビット0-7から id を取得
    unsigned newScene    = (prop >> 24) & 0xFF;
    unsigned unlocks     = (prop >> 40) & 0xFF;
    bool      isOnetime  = (prop >> 33) & 0x1;
    unsigned newRoot     = (prop >> 16) & 0xFF;

    // globalの gParent に選択要素の id を設定
    gParent = id;

    if (isOnetime) {
        for (auto& item : containerA) {
            if (item.text == selected.text && item.prop == selected.prop) {
                item.prop |= (1ULL << 34);
                break;
            }
        }
    }
}

```

```
    if (newScene != 0) {
        gScene = newScene;
        gRoot = 0;
        gParent = 0;
        gUnlocks = 0;

        processContainerLifecycle();
    } else {
        gUnlocks |= unlocks;

        if (newRoot != 0) {
            gRoot = newRoot;
        }

        processContainerLifecycle();
    }

    needRedraw = true;
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    if (!SDL_CreateWindowAndRenderer("DDMEditor Focus Window", 320, 240, 0,
        std::cerr << "Window/Renderer Creation Error: " << SDL_GetError() <<
        return SDL_APP_FAILURE;
    }

    std::string file_path = "../dd.txt";

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;
    bool is_text = true;

    while (file >> token) {
        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }
    }
}
```

```
    }
    if (clean_token.empty()) continue;

    if (is_text) {
        current_text = clean_token;
        is_text = false;
    } else {
        uint64_t prop = std::stoull(clean_token, nullptr, 16);
        containerA.push_back({current_text, prop});
        is_text = true;
    }
}

file.close();

processContainerLifecycle();

return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (key == SDLK_F) {
            sendProperty();
        } else if (!displayList.empty()) {
            size_t total = displayList.size();

            if (key == SDLK_O) {
                selectedIndex = (selectedIndex + 1) % total;
                needRedraw = true;
            } else if (key == SDLK_I) {
                selectedIndex = (selectedIndex - 1 + total) % total;
                needRedraw = true;
            }
        }
    }
}

return SDL_APP_CONTINUE;
}
```

```

SDL_AppResult SDL_AppIterate(void *appstate) {
    if (needRedraw) {
        renderConsole();
        needRedraw = false;
    }

    SDL_SetRenderDrawColor(gRenderer, 30, 30, 30, 255);
    SDL_RenderClear(gRenderer);
    SDL_RenderPresent(gRenderer);

    return SDL_APP_CONTINUE;
}

void SDL_AppQuit(void *appstate, SDL_AppResult result) {
    if (gRenderer) {
        SDL_DestroyRenderer(gRenderer);
        gRenderer = nullptr;
    }
    if (gWindow) {
        SDL_DestroyWindow(gWindow);
        gWindow = nullptr;
    }
}

```

以上の内容をデバッグできる、dd.txtを生成。

新しく追加された gParent の遷移と、要素数0件時の gParent = gRoot への
フォールバック（no content 一時表示）を動作確認・デバッグできる dd.txt
の設定例です。

デバッグ用 dd.txt

Plaintext



```

/* === 親: parent = 0 (ルート階層の動詞) === */
// [1] 見る (id=1, root=1) -> parent=1 へ遷移
'見る', '0x0000000001010001',

```

```

// [2] 話す (id=2, root=2) -> parent=2 へ遷移 (*配下に子要素がないため、選択時にフッ
'話す', '0x0000000002020002',

```

```
// [3] 行く (id=3, root=3) -> parent=3 へ遷移
'行く', '0x00000000004030003',

/* === 親: parent = 1 (「見る」を選択したとき表示される子要素) === */
// [4] a氏 (id=4, parent=1) -> 選択すると parent=4 になるが子要素0件のため no con
'a氏', '0x00010000000000104',

// [5] 公園 (id=5, parent=1) -> 選択すると parent=5 になるが子要素0件のため no cor
'公園', '0x00010000000000105',

/* === 親: parent = 3 (「行く」を選択したとき表示される子要素) === */
// [6] 公園 (id=7, parent=3) -> 選択すると parent=7 になるが子要素0件のため no cor
'公園', '0x00040000000000307'
```

デバッグ時の挙動確認ポイント

1. 正常な深掘り動作:

初期状態 (gRoot=0 , gParent=0) で 「見る」 (id=1 , newRoot=1) を選択すると、 gParent が 1 に更新され、配下の 「a氏」 「公園」 が表示されます。

2. フォールバック動作 (子要素が存在しない場合) :

上記で 「a氏」 (id=4) を選択すると、一瞬 gParent=4 を探しに行きます。

parent=4 の要素は存在しない (containerC.empty()) ため、コンソールに一時的に no content が表示された後、
gParent = gRoot (この場合は 1) に復帰し、再び 「a氏」 「公園」の一覧に戻ります。

3. 直下の子要素が最初から存在しない場合:

初期状態から 「話す」 (id=2 , newRoot=2) を選択した場合、parent=2 の要素が存在しないため、直ちに no content を挟んで gParent = gRoot (2) に遷移し、画面が更新されます。

```
terminate called after throwing an instance of
'std::invalid_argument'
what(): stoull
[1] 50831 abort (core dumped) ./DDMEditor
```

このクラッシュは、ファイル読み込み処理中の `std::stoull` が 16進数文字列ではないトークン（コメントなど）を変換しようとして失敗したことが原因です。

原因と具体的な修正案をまとめました。

原因

`dd.txt` に含まれる `/* ... */` や `//` などのコメント文や説明文が `file >> token` によって読み込まれてしまい、それが `clean_token` を通って `std::stoull` に渡されて例外が発生しています。

C++



```
// 従来の処理：コメント記号などを無視する処理がないためクラッシュする
uint64_t prop = std::stoull(clean_token, nullptr, 16);
```

解決策（`main.cpp` の修正）

ファイル読み込み処理（`SDL_AppInit` 内）を以下のように書き換えてください。

1. `/* ... */` や `//` のコメントをスキップする処理を追加
2. `std::stoull` を `try-catch` で囲み、無効な文字列を安全にスキップ

C++



```
// === SDL_AppInit 内のファイル読み込み処理を修正 ===
std::string token;
std::string current_text;
```

```

bool is_text = true;

while (file >> token) {
    // C/C++形式のコメントをスキップ
    if (token.rfind("//", 0) == 0 || token.rfind("/*", 0) == 0 || token.
        std::string dummy;
        // 行末まで無視（単一行コメントの場合）
        std::getline(file, dummy);
        continue;
    }

    // 記号を取り除く処理
    std::string clean_token = "";
    for (char c : token) {
        if (c != '\'' && c != ',' && c != ' ' && c != '\r' && c != '\n')
            clean_token += c;
    }
    if (clean_token.empty()) continue;

    if (is_text) {
        current_text = clean_token;
        is_text = false;
    } else {
        try {
            uint64_t prop = std::stoull(clean_token, nullptr, 16);
            containerA.push_back({current_text, prop});
            is_text = true;
        } catch (const std::invalid_argument&) {
            // 万が一16進数以外の文字列が混ざった場合はテキスト読み込み状態にリセット
            is_text = true;
            continue;
        } catch (const std::out_of_range&) {
            is_text = true;
            continue;
        }
    }
}
file.close();

```

テスト用 dd.txt（コメントなし完全版）

もし手っ取り早く動かしたい場合は、一度 dd.txt からコメントを取り除いた以下のデータで試してみてください。

Plaintext



```
'見る' '0x0000000001010001'
'話す' '0x0000000002020002'
'行く' '0x0000000004030003'
'a氏' '0x0001000000000104'
'公園' '0x0001000000000105'
'公園' '0x0004000000000307'
```

no content表示時は'f'入力で、最終画面を描画。

no content 表示中にキー入力をブロックせず、「no content が表示されている状態」で F キーを押すことで次の表示（containerC の最新内容）に進むように処理を修正しました。

具体的には、processContainerLifecycle() 内での強制待機（SDL_Delay）を取り払い、F キー入力をトリガーにして画面を再構築・描画する状態へ切り替えます。

修正内容のポイント

1. noContentState フラグの導入:

containerC が空になり、gParent = gRoot で再構築が必要になった際、noContentState = true とします。

この状態では displayList が空となり、画面に no content が表示されます。

2. sendProperty()（F キー入力）での状態判定:

noContentState == true の時に F キー（sendProperty()）が押された場合、gParent = gRoot に基いた rebuildDisplayList() を実行して表示リストを構築し、noContentState を解除して通常画面を描画します。

修正版プログラム (main.cpp)

C++



```
#define SDL_MAIN_USE_CALLBACKS 1
#include <SDL3/SDL.h>
#include <SDL3/SDL_main.h>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdint>
#include <random>
#include <algorithm>

struct Element {
    std::string text;
    uint64_t prop;
};

// グローバル変数
std::vector<Element> containerA;
std::vector<Element> containerB;
std::vector<Element> containerC;
std::vector<Element> displayList;

size_t selectedIndex = 0;
bool needRedraw = true;
bool noContentState = false; // no content 表示中フラグを追加

SDL_Window *gWindow = nullptr;
SDL_Renderer *gRenderer = nullptr;

unsigned gScene = 0;
unsigned gRoot = 0;
unsigned gParent = 0;
unsigned gUnlocks = 0;

void rebuildContainerB();
void rebuildContainerC();
void rebuildDisplayList();

void renderConsole() {
    std::cout << "\033[2J\033[1;1H";

    std::cout << "[ State ] gScene: " << gScene
                << ", gRoot: " << gRoot
```

```

        << ", gParent: " << gParent
        << ", gUnlocks: 0x" << std::hex << gUnlocks << std::dec << "\r\n";

    if (noContentState || displayList.empty()) {
        std::cout << "no content" << std::endl;
        std::cout << "\n(Press 'f' to continue)" << std::endl;
        return;
    }

    for (size_t i = 0; i < displayList.size(); ++i) {
        if (i == selectedIndex) {
            std::cout << "\033[31m[" << displayList[i].text << "]\033[0m ";
        } else {
            std::cout << "\033[37m[" << displayList[i].text << "]\033[0m ";
        }
    }
    std::cout << std::endl;
    std::cout << "(Focus Window -> 'o': Right, 'i': Left, 'f': Send, 'q': Quit)" << std::endl;
    std::cout << std::flush;
}

void rebuildContainerB() {
    containerB.clear();
    for (const auto& item : containerA) {
        unsigned scene = (item.prop >> 36) & 0xFF;
        if (scene == gScene) {
            containerB.push_back(item);
        }
    }
}

void rebuildContainerC() {
    containerC.clear();
    for (const auto& item : containerB) {
        uint64_t prop = item.prop;

        unsigned parent    = (prop >> 8) & 0xFF;
        unsigned hasRead    = (prop >> 34) & 0x1;
        unsigned reqFlags   = (prop >> 48) & 0xFF;
        unsigned excFlags   = (prop >> 56) & 0xFF;

        bool cond1 = (parent == gParent);
        bool cond2 = ((gUnlocks & reqFlags) == reqFlags);
        bool cond3 = (hasRead == 0);
        bool cond4 = ((gUnlocks & excFlags) == 0);

        if (cond1 && cond2 && cond3 && cond4) {
            containerC.push_back(item);
        }
    }
}

```

```

    }
}

void rebuildDisplayList() {
    displayList.clear();
    selectedIndex = 0;

    if (!containerC.empty()) {
        std::random_device rd;
        std::mt19937 gen(rd());

        bool hasOnetimeQuote = false;
        for (const auto& item : containerC) {
            bool isOnetime = (item.prop >> 33) & 0x1;
            bool isQuote = (item.prop >> 32) & 0x1;
            if (isOnetime && isQuote) {
                hasOnetimeQuote = true;
                break;
            }
        }

        if (hasOnetimeQuote) {
            std::uniform_int_distribution<size_t> dist(0, containerC.size());
            displayList.push_back(containerC[dist(gen)]);
        } else if (containerC.size() >= 5) {
            std::vector<Element> temp = containerC;
            std::shuffle(temp.begin(), temp.end(), gen);
            displayList.assign(temp.begin(), temp.begin() + 4);
        } else {
            displayList = containerC;
        }
    }
}

void processContainerLifecycle() {
    rebuildContainerB();
    rebuildContainerC();

    // 取得結果が0件の場合
    if (containerC.empty()) {
        gParent = gRoot;
        rebuildContainerC(); // gParent = gRoot でコンテナCを再構築

        noContentState = true; // no content 表示モードをセット
        displayList.clear();
    } else {
        noContentState = false;
        rebuildDisplayList();
    }
}

```

```
}
```

```
void sendProperty() {  
    // no content 表示中に 'f' が押された場合は、最終描画を行う  
    if (noContentState) {  
        noContentState = false;  
        rebuildDisplayList();  
        needRedraw = true;  
        return;  
    }  
  
    if (displayList.empty() || selectedIndex >= displayList.size()) return;  
  
    Element selected = displayList[selectedIndex];  
    uint64_t prop = selected.prop;  
  
    unsigned id          = prop & 0xFF;  
    unsigned newScene    = (prop >> 24) & 0xFF;  
    unsigned unlocks     = (prop >> 40) & 0xFF;  
    bool      isOnetime  = (prop >> 33) & 0x1;  
    unsigned newRoot     = (prop >> 16) & 0xFF;  
  
    gParent = id;  
  
    if (isOnetime) {  
        for (auto& item : containerA) {  
            if (item.text == selected.text && item.prop == selected.prop) {  
                item.prop |= (1ULL << 34);  
                break;  
            }  
        }  
    }  
  
    if (newScene != 0) {  
        gScene = newScene;  
        gRoot = 0;  
        gParent = 0;  
        gUnlocks = 0;  
  
        processContainerLifecycle();  
    } else {  
        gUnlocks |= unlocks;  
  
        if (newRoot != 0) {  
            gRoot = newRoot;  
        }  
  
        processContainerLifecycle();  
    }  
}
```

```
    needRedraw = true;
}

SDL_AppResult SDL_AppInit(void **appstate, int argc, char *argv[]) {
    if (!SDL_Init(SDL_INIT_VIDEO)) {
        std::cerr << "SDL_Init Error: " << SDL_GetError() << std::endl;
        return SDL_APP_FAILURE;
    }

    if (!SDL_CreateWindowAndRenderer("DDMEditor Focus Window", 320, 240, 0,
        std::cerr << "Window/Renderer Creation Error: " << SDL_GetError() <<
        return SDL_APP_FAILURE;
    }

    std::string file_path = "../dd.txt";

    std::ifstream file(file_path);
    if (!file.is_open()) {
        std::cerr << "エラー: " << file_path << " を開けませんでした。" << std::
        return SDL_APP_FAILURE;
    }

    std::string token;
    std::string current_text;
    bool is_text = true;

    while (file >> token) {
        if (token.rfind("//", 0) == 0 || token.rfind("/*", 0) == 0 || token.
            std::string dummy;
            std::getline(file, dummy);
            continue;
        }

        std::string clean_token = "";
        for (char c : token) {
            if (c != '\\' && c != ',' && c != ' ' && c != '\r' && c != '\n')
                clean_token += c;
        }

        if (clean_token.empty()) continue;

        if (is_text) {
            current_text = clean_token;
            is_text = false;
        } else {
            try {
                uint64_t prop = std::stoull(clean_token, nullptr, 16);
                containerA.push_back({current_text, prop});
            }
        }
    }
}
```

```
        is_text = true;
    } catch (const std::exception&) {
        is_text = true;
        continue;
    }
}

}

file.close();

processContainerLifecycle();

return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppEvent(void *appstate, SDL_Event *event) {
    if (event->type == SDL_EVENT_QUIT) {
        return SDL_APP_SUCCESS;
    }

    if (event->type == SDL_EVENT_KEY_DOWN) {
        SDL_Keycode key = event->key.key;

        if (key == SDLK_Q || key == SDLK_ESCAPE) {
            return SDL_APP_SUCCESS;
        }

        if (key == SDLK_F) {
            sendProperty();
        } else if (!displayList.empty() && !noContentState) {
            size_t total = displayList.size();

            if (key == SDLK_O) {
                selectedIndex = (selectedIndex + 1) % total;
                needRedraw = true;
            } else if (key == SDLK_I) {
                selectedIndex = (selectedIndex - 1 + total) % total;
                needRedraw = true;
            }
        }
    }

    return SDL_APP_CONTINUE;
}

SDL_AppResult SDL_AppIterate(void *appstate) {
    if (needRedraw) {
        renderConsole();
        needRedraw = false;
    }
}
```

```
SDL_SetRenderDrawColor(gRenderer, 30, 30, 30, 255);  
SDL_RenderClear(gRenderer);  
SDL_RenderPresent(gRenderer);  
  
return SDL_APP_CONTINUE;  
}
```